

```

1  const sqlite3 = require('sqlite3').verbose();
2  const path = require('path');
3
4  const dbPath = path.resolve(__dirname, 'healthcare.db');
5  const db = new sqlite3.Database(dbPath, (err) => {
6    if (err) {
7      console.error('Error opening SQLite database:', err.message);
8    } else {
9      console.log('Connected to the local SQLite database.');
```

```

10     initializeDatabase();
11   }
12 });
13
14 const crypto = require('crypto');
```

```

15
16 function hashPassword(password, salt) {
17   return crypto.pbkdf2Sync(password, salt, 1000, 64, 'sha512').toString('hex');
```

```

18 }
19
20 function generateSalt() {
21   return crypto.randomBytes(16).toString('hex');
```

```

22 }
23
24 function initializeDatabase() {
25   db.serialize(() => {
26     // Create users table
27     db.run(`
28       CREATE TABLE IF NOT EXISTS users (
29         id INTEGER PRIMARY KEY AUTOINCREMENT,
30         username TEXT UNIQUE NOT NULL,
31         email TEXT UNIQUE NOT NULL,
32         password_hash TEXT NOT NULL,
33         salt TEXT NOT NULL,
34         role TEXT NOT NULL CHECK(role IN ('Admin', 'Staff', 'Patient')),
35         created_at DATETIME DEFAULT CURRENT_TIMESTAMP
36       )
37     `);
38
39     // Create appointments table with user_id
40     db.run(`
41       CREATE TABLE IF NOT EXISTS appointments (
42         id INTEGER PRIMARY KEY AUTOINCREMENT,
43         user_id INTEGER,
44         patient_name TEXT NOT NULL,
45         email TEXT NOT NULL,
46         phone TEXT NOT NULL,
47         appointment_date TEXT NOT NULL,
48         appointment_time TEXT NOT NULL,
49         status TEXT DEFAULT 'pending',
50         notes TEXT,
51         created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
52         FOREIGN KEY(user_id) REFERENCES users(id)
53       )
54     `);
55
56     // Attempt to alter appointments table to add user_id in case it was created without
    it previously
57     db.run("ALTER TABLE appointments ADD COLUMN user_id INTEGER", (err) => {
58       // Ignore error if column already exists
59     });
60
61     // Create news table
62     db.run(`
63       CREATE TABLE IF NOT EXISTS news (
64         id INTEGER PRIMARY KEY AUTOINCREMENT,
65         title TEXT NOT NULL,
66         content TEXT NOT NULL,
67         image_url TEXT,
68         category TEXT NOT NULL,
```

```

69     date_posted DATETIME DEFAULT CURRENT_TIMESTAMP
70 )
71 `);
72
73 // Seed default users if the table is empty
74 db.get("SELECT COUNT(*) as count FROM users", [], (err, row) => {
75     if (err) return console.error(err.message);
76     if (row.count === 0) {
77         const stmt = db.prepare("INSERT INTO users (username, email, password_hash,
78             salt, role) VALUES (?, ?, ?, ?, ?)");
79
80         // Admin user
81         const adminSalt = generateSalt();
82         stmt.run("admin", "admin@alamnagar-chc.org", hashPassword("adminpass",
83             adminSalt), adminSalt, "Admin");
84
85         // Staff user
86         const staffSalt = generateSalt();
87         stmt.run("staff", "staff@alamnagar-chc.org", hashPassword("staffpass",
88             staffSalt), staffSalt, "Staff");
89
90         // Patient user
91         const patientSalt = generateSalt();
92         stmt.run("patient", "patient@example.com", hashPassword("patientpass",
93             patientSalt), patientSalt, "Patient");
94
95         stmt.finalize();
96         console.log("Seeded default users (admin, staff, patient).");
97     }
98 });
99
100 // Insert initial news items if table is empty
101 db.get("SELECT COUNT(*) as count FROM news", [], (err, row) => {
102     if (err) return console.error(err.message);
103     if (row.count === 0) {
104         const stmt = db.prepare("INSERT INTO news (title, content, image_url, category)
105             VALUES (?, ?, ?, ?)");
106         stmt.run(
107             "Free Medical Health Camp Next Saturday",
108             "Alamnagar Charitable Healthcare Centre is organizing a free health check-up
109             camp next Saturday. General physicians, pediatricians, and cardiologists will
110             be available for consultations from 9:00 AM to 3:00 PM. Free medicine
111             distribution is also arranged.",
112             "https://images.unsplash.com/photo-1576091160550-2173dba999ef?auto=format&fit=c
113             rop&w=600&q=80",
114             "Event"
115         );
116         stmt.run(
117             "New Pediatric Specialist Joins Our Team",
118             "We are pleased to welcome Dr. Sarah Rahman, MD in Pediatrics, to our medical
119             team. She will be available for consultations every Monday and Wednesday
120             starting next week. Book your appointments online.",
121             "https://images.unsplash.com/photo-1559839734-2b71ea197ec2?auto=format&fit=crop
122             &w=600&q=80",
123             "News"
124         );
125         stmt.run(
126             "COVID-19 Booster Dose Guidelines",
127             "We are offering booster doses of COVID-19 vaccines for senior citizens and
128             high-risk patients. Walk-ins are welcome from 10:00 AM to 2:00 PM on weekdays.
129             Please bring your previous vaccination records.",
130             "https://images.unsplash.com/photo-1584515979956-d9f6e5d09982?auto=format&fit=c
131             rop&w=600&q=80",
132             "Alert"
133         );
134         stmt.finalize();
135     }
136 });

```

```

120         console.log("Inserted initial news items.");
121     }
122     });
123 });
124 }
125
126 // Database query helpers
127 module.exports = {
128     db,
129     hashPassword,
130     generateSalt,
131
132     // User operations
133     getUserByUsername: (username) => {
134         return new Promise((resolve, reject) => {
135             db.get("SELECT * FROM users WHERE username = ?", [username], (err, row) => {
136                 if (err) reject(err);
137                 else resolve(row);
138             });
139         });
140     },
141
142     getUserByEmail: (email) => {
143         return new Promise((resolve, reject) => {
144             db.get("SELECT * FROM users WHERE email = ?", [email], (err, row) => {
145                 if (err) reject(err);
146                 else resolve(row);
147             });
148         });
149     },
150
151     createUser: (user) => {
152         return new Promise((resolve, reject) => {
153             const { username, email, password, role } = user;
154             const salt = generateSalt();
155             const hash = hashPassword(password, salt);
156             const query = `INSERT INTO users (username, email, password_hash, salt, role)
157                 VALUES (?, ?, ?, ?, ?)`;
158
159             db.run(query, [username, email, hash, salt, role], function(err) {
160                 if (err) reject(err);
161                 else resolve({ id: this.lastID, username, email, role });
162             });
163         });
164     },
165
166     // Appointments operations
167     getAllAppointments: () => {
168         return new Promise((resolve, reject) => {
169             db.all("SELECT * FROM appointments ORDER BY appointment_date ASC, appointment_time
170                 ASC", [], (err, rows) => {
171                 if (err) reject(err);
172                 else resolve(rows);
173             });
174         });
175     },
176
177     getAppointmentsByUserId: (userId) => {
178         return new Promise((resolve, reject) => {
179             db.all("SELECT * FROM appointments WHERE user_id = ? ORDER BY appointment_date
180                 ASC, appointment_time ASC", [userId], (err, rows) => {
181                 if (err) reject(err);
182                 else resolve(rows);
183             });
184         });
185     },
186
187     createAppointment: (appointment) => {
188         return new Promise((resolve, reject) => {

```

```

186     const { user_id, patient_name, email, phone, appointment_date, appointment_time,
187           notes } = appointment;
188     const query = `INSERT INTO appointments (user_id, patient_name, email, phone,
189           appointment_date, appointment_time, notes)
190           VALUES (?, ?, ?, ?, ?, ?, ?)`;
191     db.run(query, [user_id || null, patient_name, email, phone, appointment_date,
192           appointment_time, notes], function(err) {
193       if (err) reject(err);
194       else resolve({ id: this.lastID, ...appointment, status: 'pending' });
195     });
196   });
197 },
198
199 updateAppointmentStatus: (id, status) => {
200   return new Promise((resolve, reject) => {
201     const query = `UPDATE appointments SET status = ? WHERE id = ?`;
202     db.run(query, [status, id], function(err) {
203       if (err) reject(err);
204       else resolve({ changes: this.changes });
205     });
206   });
207 },
208
209 // News operations
210 getAllNews: () => {
211   return new Promise((resolve, reject) => {
212     db.all("SELECT * FROM news ORDER BY date_posted DESC", [], (err, rows) => {
213       if (err) reject(err);
214       else resolve(rows);
215     });
216   });
217 },
218
219 createNews: (newsItem) => {
220   return new Promise((resolve, reject) => {
221     const { title, content, image_url, category } = newsItem;
222     const query = `INSERT INTO news (title, content, image_url, category) VALUES (?,
223           ?, ?, ?)`;
224     db.run(query, [title, content, image_url, category], function(err) {
225       if (err) reject(err);
226       else resolve({ id: this.lastID, ...newsItem, date_posted: new
227             Date().toISOString() });
228     });
229   });
230 }
231 };

```